From Syntax to Problem Solving

A Practical Coding Practice Workbook for Engineering Students

Learn to understand the problem, build the logic, test your solution, and improve from mistakes.

Who this is for

Engineering students who know basic programming syntax in Python, Java, C++, or another language, but find it difficult to begin solving unfamiliar problems independently.

How to use this workbook

Write your reasoning before code. Use the blank spaces. Test your first attempt. Open the answer key only after you have made a genuine attempt and can describe where you are stuck.

ATTEMPT FIRST	TRACE YOUR LOGIC	TEST EDGE CASES

Creator / Instagram: _____

1. A repeatable problem-solving framework

Use the same sequence whenever a problem feels unfamiliar. The goal is to make the first step clear, not to guess the final code immediately.

1. Understand the question

Restate the task in your own words. Identify exactly what must be calculated or decided.

2. Identify input and output

List the data you receive and the result you must return or print. Record constraints and assumptions.

3. Work through a small example

Solve one example by hand. Choose values that make each step visible.

4. Develop the logic

Ask what information must be tracked and how it changes as you process the input.

5. Write pseudocode

Pseudocode is a language-neutral list of steps. It lets you check the algorithm before worrying about syntax.

6. Implement the solution

Translate each step into your chosen language. Keep variable names meaningful.

7. Test normal and edge cases

Try an ordinary case, a smallest valid case, repeated values, and any special boundary allowed by the statement.

8. Review and improve

If it fails, find the earliest step where actual behaviour differs from expected behaviour. Record the lesson.

Worked mini-example: Find the largest number

Problem: Given a non-empty list of integers, return its largest value.

Input: [4, 9, 2, 7] **Expected output:** 9

Reasoning: Start by treating the first value (4) as the largest seen. Compare each later value with it. Replace the current largest only when a larger value appears.

```
largest = first value
for each remaining value:
    if value > largest:
        largest = value
return largest
```

Dry run: A dry run means tracing variable values manually, one step at a time.

Value read	Largest before	Comparison	Largest after
4	—	Initial value	4
9	4	9 > 4	9
2	9	2 > 9 is false	9
7	9	7 > 9 is false	9

The assumption “non-empty list” matters because there is no first value to initialise the largest variable if the list is empty.

2. Guided example: Find the largest number

Problem statement

Given a non-empty array of integers, find and return the largest element. Do not sort the array; inspect its values directly.

Define the problem

Question	Answer
What is given?	A non-empty array of integers.
What must be returned?	One integer: the largest value.
Can values be negative?	Yes. No restriction excludes negative integers.
What if all values are equal?	That repeated value is the largest.
What if the array is empty?	Outside this problem's stated assumptions; empty input is not accepted.

Manual reasoning

For [–3, 5, 1, 5, 2], begin with –3 as the largest. Read 5 and update the largest to 5. The next values (1, 5, 2) are not greater than 5, so the answer stays 5.

Pseudocode

```
largest = numbers[0]
for each number in numbers starting at index 1:
    if number > largest:
        largest = number
return largest
```

Trace table

Index	Current value	Largest before	Update?	Largest after
0	-3	—	Initialise	-3
1	5	-3	Yes	5
2	1	5	No	5
3	5	5	No (equal)	5
4	2	5	No	5

Python implementation

```
def largest_number(numbers):
    # Assumption: numbers is not empty.
    largest = numbers[0]
    for number in numbers[1:]:
        if number > largest:
            largest = number
    return largest

print(largest_number([-3, 5, 1, 5, 2]))  # 5
```

Important lines: Initialising from numbers[0] works even when all values are negative. The comparison uses >, so equal values do not trigger an unnecessary update. The function returns a value instead of printing from inside the loop.

Time complexity: O(n), read as “linear time.” For n numbers, the algorithm examines each value at most once. It uses O(1) extra space because it tracks only one additional value.

Test: [8] → 8; [-8, -2, -11] → -2; [4, 4, 4] → 4; [1, 9, 3] → 9.

3. Find the largest of three numbers

Given three integers, return the largest number. Do not use a built-in maximum function.

Input / output: Three integers a, b, c → one integer.

Examples: a=4, b=9, c=2 → 9; a=-5, b=-2, c=-8 → -2.

Constraints / assumptions: Assume all three inputs are valid integers. Values may be negative or equal.

Skill focus: Comparisons and conditional logic.

A. My reasoning — explain the steps before coding

B. Pseudocode — write ordered steps

C. Dry run — trace at least one example

Step / value	Variable(s) before	Action	Result after

D. Code and testing checklist

Code area: _____

_____

■ Normal case ■ Smallest valid input ■ Boundary / repeated values ■ Output matches the statement

If stuck: Track a current largest value, then compare the other two values against it.

4. Even or odd

Given an integer n, determine whether it is even or odd.

Input / output: One integer n → the text Even or Odd.

Examples: n=14 → Even; n=-3 → Odd.

Constraints / assumptions: n may be positive, zero, or negative. An integer is even if it is divisible by 2 with remainder 0.

Skill focus: Remainder operator and conditionals.

A. My reasoning — explain the steps before coding

B. Pseudocode — write ordered steps

C. Dry run — trace at least one example

Step / value	Variable(s) before	Action	Result after

D. Code and testing checklist

Code area: _____

_____

■ Normal case ■ Smallest valid input ■ Boundary / repeated values ■ Output matches the statement

If stuck: Consider what n % 2 tells you. Test zero and a negative odd number.

5. Sum the numbers in an array

Given an array of integers, calculate the sum of all its elements.

Input / output: A non-empty list of integers → one integer sum.

Examples: [2, 4, 6] → 12; [-2, 5] → 3.

Constraints / assumptions: Array contains at least one integer. Values may be negative or zero.

Skill focus: Looping and accumulator variables. An accumulator stores a running result.

A. My reasoning — explain the steps before coding

B. Pseudocode — write ordered steps

C. Dry run — trace at least one example

Step / value	Variable(s) before	Action	Result after

D. Code and testing checklist

Code area: _____

_____

■ Normal case ■ Smallest valid input ■ Boundary / repeated values ■ Output matches the statement

If stuck: Start a total at 0, then add each element exactly once.

6. Count vowels in a string

Count the English vowels a, e, i, o, u in a string. Count uppercase and lowercase vowels. Do not count y as a vowel.

Input / output: One string → a non-negative integer count.

Examples: "Education" → 5; "rhythm" → 0.

Constraints / assumptions: The string may be empty and may contain spaces or punctuation.

Skill focus: String traversal, case normalisation, and counting.

A. My reasoning — explain the steps before coding

B. Pseudocode — write ordered steps

C. Dry run — trace at least one example

Step / value	Variable(s) before	Action	Result after

D. Code and testing checklist

Code area: _____

_____

■ Normal case ■ Smallest valid input ■ Boundary / repeated values ■ Output matches the statement

If stuck: Process one character at a time. Decide how to treat uppercase letters before testing membership.

7. Second-largest distinct element

Given a non-empty array of integers, return the second-largest distinct value. If no such value exists, return None.

Input / output: A list of integers → an integer or None.

Examples: [4, 1, 7, 7, 3] → 4; [5, 5] → None.

Constraints / assumptions: Duplicates do not count as a second distinct value. Values may be negative.

Skill focus: Maintaining two candidate values and handling duplicates.

A. My reasoning — explain the steps before coding

B. Pseudocode — write ordered steps

C. Dry run — trace at least one example

Step / value	Variable(s) before	Action	Result after

D. Code and testing checklist

Code area: _____

_____

■ Normal case ■ Smallest valid input ■ Boundary / repeated values ■ Output matches the statement

If stuck: Ask whether each value is larger than the current largest, or lies strictly between largest and second-largest.

8. Palindrome check

Return True if a string reads the same forward and backward; otherwise return False. Ignore letter case, but do not remove spaces or punctuation.

Input / output: One string → True or False.

Examples: "Level" → True; "hello" → False; "Never odd" → False.

Constraints / assumptions: The empty string is considered a palindrome. Spaces and punctuation are compared as characters.

Skill focus: String indexing and symmetric comparison.

A. My reasoning — explain the steps before coding

B. Pseudocode — write ordered steps

C. Dry run — trace at least one example

Step / value	Variable(s) before	Action	Result after

D. Code and testing checklist

Code area: _____

_____

■ Normal case ■ Smallest valid input ■ Boundary / repeated values ■ Output matches the statement

If stuck: Compare the first and last characters, then move inward. Alternatively, reason about reversing the string.

9. Debugging and edge-case checklist

When output is wrong, do not randomly rewrite the program. Reproduce the failure, shrink the input, and inspect variable values one step at a time. Syntax errors prevent code from being parsed; logic errors allow code to run but produce the wrong result. [3]

Before changing code, ask:

- ■ Did I restate the task correctly, including the required output?
- ■ Does the code handle the smallest valid input?
- ■ What happens when values are repeated, negative, or already sorted?
- ■ Did I make an off-by-one error (starting or stopping one position too early or late)?
- ■ Did I update the intended variable, and is it initialised correctly?
- ■ Does the loop run the intended number of times?
- ■ Am I returning the final answer rather than an intermediate result?
- ■ Can I reproduce the bug with a smaller test case?
- ■ Have I compared actual output with the expected output?

Debugging exercise — identify the bug before reading the answer key

The function should return the sum of every number in the list. Test it with [2, 4, 6].

```
def sum_numbers(numbers):
    total = 0
    for i in range(1, len(numbers)):
        total += numbers[i]
    return total

print(sum_numbers([2, 4, 6]))  # Expected: 12
```

What output does this code actually produce? Which element is skipped, and which loop boundary should change?

Actual output: _____ Bug: _____

Correction in your own words:

Debugging sequence: (1) reproduce the failure, (2) predict the next variable value, (3) trace one loop iteration at a time, (4) make one targeted change, (5) rerun the original and boundary tests. This mirrors the iterative approach demonstrated in CS50 debugging materials. [3, 4]

10. Practise without becoming dependent on solutions

Use this as a suggested 30-minute routine, not a scientifically proven optimum. Shorten or extend each phase depending on the problem and your current level.

Time	Action	What to produce
0–3 min	Read the statement twice. Mark input, output, constraints, and unclear terms.	One-sentence restatement + assumptions.
3–8 min	Work through a small example manually. Try a boundary case too.	A small trace table.
8–16 min	Write logic or pseudocode, then attempt implementation.	A plan and a first solution.
16–21 min	Run tests. If stuck, isolate the smallest failing case and write down the question.	Expected vs actual output; specific question.
21–26 min	Use one hint or study a reference solution if needed. Compare its reasoning with yours.	The key insight and what your approach missed.
26–30 min	Close the reference. Rebuild the solution from a blank editor and record a retry date.	Independent reimplementation + review note.

Use hints in layers

1. Re-read the statement and examples. 2. Try a smaller input. 3. Ask for a hint about the next reasoning step, not the full code. 4. If you read a complete solution, explain why each step works. 5. Close it and implement independently. If you still cannot reproduce it, return to the exact step you do not understand.

Review template — fill this in after each problem

Prompt	Your notes
Problem:	
My first approach:	
Where I got stuck:	
The key insight:	
My mistake:	
What I learned:	
When I will retry it:	

Revisit problems you struggled with after a gap. Try to solve them without opening your previous code first. The purpose is to check whether you can reconstruct the reasoning, not merely recognise a familiar answer.

11. Answer key and explanations

Attempt the practice pages before using this section. Solutions are written for clarity rather than minimum line count.

Exercise 3 — Largest of three

```
def largest_of_three(a, b, c):
    largest = a
    if b > largest:
        largest = b
    if c > largest:
        largest = c
    return largest

print(largest_of_three(4, 9, 2))  # 9
```

Start with a as the current largest, then compare b and c. Equal values are handled because the existing largest remains valid. Test (7, 7, 2) → 7 and (-5, -2, -8) → -2.

Exercise 4 — Even or odd

```
def even_or_odd(n):
    if n % 2 == 0:
        return "Even"
    return "Odd"

print(even_or_odd(-3))  # Odd
```

The remainder after division by 2 is zero for even integers, including zero and negative even integers. Test 0 → Even and -4 → Even.

Exercise 5 — Sum an array

```
def sum_numbers(numbers):
    total = 0
    for number in numbers:
        total += number
    return total

print(sum_numbers([2, 4, 6]))  # 12
```

The accumulator begins at 0, the additive identity, and each element is added once. Time complexity is O(n). Test [0] → 0 and [-2, 5] → 3.

Exercise 6 — Count vowels

```
def count_vowels(text):
    vowels = "aeiou"
    count = 0
    for character in text.lower():
        if character in vowels:
            count += 1
    return count

print(count_vowels("Education"))  # 5
```

Converting to lowercase makes uppercase and lowercase vowels behave consistently. Empty strings return 0; "rhythm" → 0.

Exercise 7 — Second-largest distinct value

```
def second_largest_distinct(numbers):
    largest = None
    second = None

    for number in numbers:
        if largest is None or number > largest:
            second = largest
            largest = number
        elif number != largest and (
            second is None or number > second
        ):
            second = number

    return second

print(second_largest_distinct([4, 1, 7, 7, 3]))  # 4
print(second_largest_distinct([5, 5]))            # None
```

The function tracks the largest and second-largest distinct values. A duplicate of the largest is ignored by the elif condition. If the array has fewer than two distinct values, second remains None. Test [-8, -2, -5] → -5. Time complexity is O(n).

Exercise 8 — Palindrome

```
def is_palindrome(text):
    text = text.lower()
```

```
    left = 0
    right = len(text) - 1

    while left < right:
        if text[left] != text[right]:
            return False
        left += 1
        right -= 1
    return True

print(is_palindrome("Level"))  # True
print(is_palindrome("hello"))  # False
```

Compare matching positions from the outside moving inward. This version follows the stated rule: case is ignored, but spaces and punctuation are not removed. The empty string returns True. Time complexity is O(n).

Debugging exercise

```
def sum_numbers(numbers):
    total = 0
    for i in range(len(numbers)):
        total += numbers[i]
    return total
```

The original loop began at index 1, so it skipped numbers[0]. Starting at index 0 processes every element. For [2, 4, 6], the corrected result is 12.

12. Progress tracker and next steps

Track what you can do independently, not just whether a problem eventually passed.

Problem	Attempted independently?	Solved?	Main mistake	Retry date
Largest of three				
Even or odd				
Sum array				
Count vowels				
Second-largest distinct				
Palindrome				

Before moving to harder problems

- ■ I can restate the task and identify input and output.
- ■ I can produce pseudocode before implementation.
- ■ I can dry-run the logic and explain variable changes.
- ■ I tested normal, smallest-valid, and relevant edge cases.
- ■ I can explain the bug I fixed, not just the final code.
- ■ I scheduled a retry for problems I needed help with.

What to do next — free learning resources

Python Tutorial — Python Software Foundation
https://docs.python.org/3/tutorial/
Useful for reviewing control flow, loops, functions, and core language behaviour. It assumes some basic programming knowledge. [1]

CS50x Debugging Practice — Harvard University
https://cs50.harvard.edu/x/practice/debug/
A practical exercise in reading compiler errors, fixing issues iteratively, and checking program output. [3]

HackerRank Interview Preparation Kit
https://www.hackerrank.com/interview/preparation-kits
A structured collection of coding challenges, including warm-up problems, arrays, and string manipulation. [5]

LeetCode Study Plans
https://leetcode.com/studyplan/
A catalogue of guided study plans; select a beginner-appropriate plan and focus on explaining each solution. [6]

Further learning and references

[1] Python Software Foundation, *The Python Tutorial* — control flow, loops, functions, and language fundamentals. https://docs.python.org/3/tutorial/

[2] Microsoft Careers, *Technical interviews* — breaking down problems, clarifying ambiguity, planning before implementation, and testing boundary conditions. https://careers.microsoft.com/v2/global/en/hiring-tips/technical-interviewing

[3] Harvard University, CS50x, *Debug* — syntax errors, compiler messages, iterative fixes, and checking outputs. https://cs50.harvard.edu/x/practice/debug/

[4] Harvard University, CS50x, *Lecture 2: Arrays* — tracing code, inspecting variable values, and using a debugger step by step. https://cs50.harvard.edu/x/notes/2/

[5] HackerRank, *Interview Preparation Kit* — structured practice across warm-up, arrays, and strings. https://www.hackerrank.com/interview/preparation-kits

[6] LeetCode, *Study Plan* — guided practice plans. https://leetcode.com/studyplan/

The framework and worksheet prompts are original instructional material informed by these resources. The six practice exercises are standard beginner-level patterns selected for this workbook; they are not represented as company-specific interview questions.